Sanjeevani Scan AI: Complete Methodology, Algorithms, and Tech Stack Explanation

This document provides a comprehensive, easy-to-understand explanation of every algorithm and technology used in the "Sanjeevani Scan AI" project. It also explains **why** these specific tools were chosen over their alternatives.

(Note: Your presentation slides are highly accurate, well-structured, and perfectly align with this explanation. You are good to go for the presentation!)

---

1. Type of Model: Convolutional Neural Network (CNN)

What it is: A deep learning model specifically designed for processing structured grid data like images. It uses mathematical operations called "convolutions" to slide over an image and detect patterns (like edges, curves, and eventually complex leaf shapes). **Why we used it:** CNNs are the gold standard for image classification. They automatically learn spatial hierarchies of features, meaning we don't have to manually code what a "leaf vein" looks like. **Why not others (like SVM or Random Forest):** Traditional machine learning algorithms (like SVM) require manual feature extraction (HOG, SIFT) which is incredibly difficult for complex natural objects like leaves and yields much lower accuracy.

---

2. Core Algorithms & Architectures

A. EfficientNetV2-S (The AI Backbone)

What it is: Developed by Google, EfficientNet is a family of image classification models. The "V2" indicates the second generation, and "S" stands for Small. It optimizes training speed and parameter efficiency using a combination of neural architecture search and scaling. **Why we used it:** It gives state-of-the-art accuracy with significantly fewer parameters. A smaller model size is critical because our ultimate goal is to deploy this on a mobile device (offline inference). **Why not ResNet50 or VGG16:** ResNet and VGG are older, much heavier, and slower. They consume a lot of memory and battery, making them terrible choices for lightweight mobile deployment.

B. Transfer Learning (The Training Strategy)

What it is: Instead of training our AI from scratch (a blank brain), we take a model (EfficientNet) that has already been trained on millions of everyday images (ImageNet-1K). The model already knows how to detect edges, colors, and textures. We just "transfer" this knowledge and teach it to focus specifically on Ayurvedic leaves. **Why we used it:** Training from scratch would require hundreds of thousands of leaf images and weeks of GPU computing. Transfer learning allows us to achieve ≥87% accuracy with just 35,000 images in a few hours.

C. Grad-CAM (Explainable AI / XAI)

What it is: Gradient-weighted Class Activation Mapping. It looks at the gradients flowing into the final convolutional layer to produce a coarse localization map (a heatmap) highlighting the important regions in

the image. **Why we used it:** In the medical and Ayurvedic field, trust is paramount. Grad-CAM visually proves *why* the AI made a decision (e.g., highlighting a specific serrated edge of a Neem leaf). It proves the AI isn't just guessing based on the background color. **Why not LIME or SHAP:** LIME and SHAP are computationally expensive because they require running the model multiple times to generate an explanation. Grad-CAM generates the heatmap in a single forward/backward pass, making it fast enough for real-time mobile use.

D. Perceptual Hashing (pHash)

What it is: An algorithm used during our data cleaning phase. It generates a "fingerprint" of an image based on its visual structure. **Why we used it:** We collected datasets from multiple sources (Kaggle, APIs). Many datasets contain the exact same images but resized or compressed. pHash detects these visual duplicates and removes them, preventing "data leakage" (where the same image ends up in both training and testing sets, faking high accuracy). **Why not MD5 or SHA256:** MD5 changes completely if even a single pixel's color changes by 1 value. pHash remains the same even if the image is resized, slightly cropped, or compressed.

---

3. Technology Stack: Why These Tools?

A. Deep Learning Framework: PyTorch

What it is: A modern, Python-based deep learning library developed by Meta (Facebook). **Why we used it:** PyTorch is currently the industry standard for AI research and computer vision. It is highly flexible, easy to debug, and has a massive ecosystem (like `timm` for computer vision models). **Why not TensorFlow/Keras:** While TensorFlow is good, Keras is often too high-level, making it difficult to implement custom architectures or extract intermediate layers (which is strictly required for Grad-CAM).

B. Image Preprocessing: rembg (U2Net) & Albumentations

What it is:

- `rembg` uses a neural network (U2Net) to automatically remove the background from images.
- `Albumentations` is a fast image augmentation library. **Why we used them:** `rembg` removes noisy backgrounds (like mud or hands) so the model only learns the leaf. `Albumentations` artificially expands our dataset by rotating, flipping, and changing the brightness of images, preventing the model from memorizing the training data (overfitting). **Why not Standard OpenCV / TorchVision:** Traditional OpenCV background removal (like GrabCut) requires manual tuning and fails on complex field images. Albumentations is significantly faster and offers more realistic augmentations than standard TorchVision transforms.

C. Deployment & Backend: FastAPI & Uvicorn

What it is: FastAPI is a modern, high-performance web framework for building APIs with Python. **Why we used it:** It is incredibly fast, natively supports asynchronous code (`async/await`), and automatically generates API documentation. It is the perfect bridge to serve our PyTorch/ONNX model predictions to a mobile app or web frontend. **Why not Flask or Django:** Django is too heavy and complex for a simple AI inference API. Flask is older and significantly slower because it lacks native asynchronous support.

D. Model Quantization: ONNX & TensorFlow Lite (TFLite)

What it is: ONNX (Open Neural Network Exchange) is a format to transfer models between frameworks. TFLite is a lightweight version of TensorFlow designed specifically for mobile devices. **Why we used it:** We convert our PyTorch model to ONNX, then to TFLite. During this, we apply **Quantization**, which converts the heavy 32-bit float math of the AI into lightweight 8-bit integers. This shrinks the model size to under 20MB without losing significant accuracy. **Why not deploy PyTorch directly on mobile:** Full PyTorch models are massive (often 100MB+) and consume too much RAM and battery. TFLite is hardware-accelerated for Android and iOS processors.

---

4. Final Summary: What Will We Achieve?

By combining EfficientNetV2, Grad-CAM, and TFLite, we are achieving something highly unique: We are taking **complex Ayurvedic botanical knowledge** and putting it inside a **lightweight, offline, real-time mobile application**.

Instead of relying on rare experts or internet connections, anyone can point their phone at a leaf, instantly know what species it is, receive visual proof of *why* the AI thinks so, and read its Ayurvedic properties—completely offline and error-free.